RMSNorm: Forward, Backward and Intermidiates

Core idea

- RMSNorm is applied **independently to each token vector**. (To each S in [B,S,H])
- It reduces over the **hidden dimension only**.
- Unlike LayerNorm, RMSNorm **does not subtract the mean**. It only normalizes the **scale**. Proved to maintain effectiveness. Implication : RMSNorm(cx) ~ RMSNorm(x) but RMSNorm(x+c) !~ RMSNorm(x)

Forward pass

- GammaY belongs to R^{H}

- For one token row $x \in \mathbb{R}^H$:

$$m = \frac{1}{H} \sum_i x_i^2$$

$$s = \frac{1}{\sqrt{m + \epsilon}}$$

$$y_i = \gamma_i \, x_i \, s$$

Backward pass

- Let $g = \frac{\partial L}{\partial y}$
- Define:

$$u = g \odot \gamma$$

$$c = \frac{1}{H} \sum_i x_i u_i$$

Then:

$$dx = s\,u - s^3\,x\,c$$

$$d\gamma_i = \sum_{\text{tokens}} g_i \, x_i \, s$$

What this means

- Forward needs **one row-wise reduction** of x^2 (If we compress [B,S,H] to [B*S,H] we can see row wise reduction)
- Backward needs:
  - **one row-wise dot product** for dx (for c)
  - **one column-wise reduction** for dγ

Implementation note

- In practice, the reduction for mmm is usually done in **fp32** for numerical stability, even if input/output are bf16.

Pytorch implementation

torch/nn/functional.py->native_functions.yaml->laten/src/ATen/native/layer_norm.cpp ->aten/src/ATen/native/cuda/layer_norm_kernel.cu (for fused version) - have a lot of shared template between layernorm and rmsnorm

Implementation

- A fused path and a non fused path written in CUDA.
- Non fused path for RMS taken when dtype mismatch between weights and input. (Common scenario: Under torch.autocast(bf16), if your nn.RMSNorm was constructed in fp32 (which is the default) and the input comes in as bf16 (because the preceding linear's output got autocasted), **the fused kernel silently won't fire** and you fall back to composite)
- PyTorch's autograd for RMSNorm is **not** a torch.autograd.Function. It's registered declaratively in tools/autograd/derivatives.yaml (C++ implementation to achieve the same thing what torch,autograd.Function does i.e to run forward, save context and call respective backward)

  Derivatives.yaml is used for all the built in Pytorch implementations (arnd 2000+) since a yaml file is more compact and faster (since no python overhead) instead of using Python implementation. Python implementation is user API.

- Second, the backward has **two branches**:
  - If GradMode::is_enabled() (you're inside another backward — i.e., double backward) or the gradient of rstd itself is needed: use infinitely_differentiable_native_rms_norm_backward — this is a composite implementation built from differentiable ops, so autograd can traverse it again.
  - Otherwise (the common case, single backward): call the fused CUDA kernel _fused_rms_norm_backward

Additional Memory

Forward:

Forward, fused path — what persists in HBM after the op returns:

Tensor	Size	Notes
`X` (input)	$M \times N$	Already existed; saved for backward
`Y` (output)	$M \times N$	Newly allocated
`rstd`	M (fp32 accum type)	One scalar per row; saved for backward
`weight`	N	Already existed

-Fused path: Y ~ 4096*4096 in bf16 ~ 32MB, rstd ~ 16KB

-For Unfused path: the working transient memory is 4-5x of input memory

Backward:

- dX, dgammaY, part_grad_gamma

-**waht is part_grad_gamma?** :It's an **intermediate tensor** that exists only during the backward pass, gets used to compute dweight, then gets freed. Let me explain why it exists.

Why you need scratch in the first place. Computing dweight requires summing across all M=4096 rows. If you try to do that sum in one go with one thread block per column, you get terrible memory access patterns (striding through a row-major matrix). If you try to do it with atomic adds (every row's thread block atomically adds into dweight[n]), you get contention — 4096 blocks fighting for 4096 memory locations.

So the kernel does a **two-stage reduction**:

- **Stage 1:** Split the M rows into chunks. Each chunk handles rows_per_block rows (say, 32). Each chunk computes a *partial* sum and writes it to its own slot in a scratch buffer. No contention, no atomics.
- **Stage 2:** A tiny second kernel sums the partial sums down to the final dweight

How its being used in other LLMs

-Modern decoder-only LLMs use RMSNorm in three places, all **pre-norm**.

- A 32-layer model has **65 RMSNorm calls per forward pass** (32 pre-attn + 32 pre-MLP + 1 final just before lm_head)
- Point to be noted though is that almost every HuggingFace model ships its own Python RMSNorm, not nn.RMSNorm. Its because RMSNorm came very late to HF therefore every model that wanted to use it made their own class and now after the checkpoints and model config has been released, renaming the class or using nn.RMSNorm would break backward compatibility. Most are copies of LLama RMSNorm implementation, some RMSNorms also tweak the implementation a bit.

  For eg LLamaRMSNorm() – one implementation detail about this is that it always forces the naive path in pytorch due to casting.

Standard ~ LLama Implementation of RMSNorm

Model family	Class name	Variant
Llama / Llama-2 / Llama-3 / Llama-4	`LlamaRMSNorm`	Standard
Mistral / Mixtral	`MistralRMSNorm`	Standard (copy of Llama)
Qwen / Qwen2 / Qwen2.5	`Qwen2RMSNorm`	Standard
Qwen2-MoE	`Qwen2MoeRMSNorm`	Standard
Qwen3 / Qwen3-MoE / Qwen3-VL	`Qwen3RMSNorm`	Standard, plus used for QK-norm on head_dim
Gemma / Gemma-2	`GemmaRMSNorm`	`x * (1 + weight)` variant, weight init to 0
Gemma-3	`Gemma3RMSNorm`	Same Gemma variant
Phi-3	`Phi3RMSNorm`	Standard
DeepSeek / DeepSeek-V2 / V3	`DeepseekV2RMSNorm` etc.	Standard
Yi	`YiRMSNorm`	Standard (Llama clone)
InternLM / InternLM2	`InternLM2RMSNorm`	Standard
Falcon-Mamba	`FalconMambaRMSNorm`	Standard
T5	`T5LayerNorm`	RMSNorm without scale centering — the original RMSNorm
OLMo / OLMo2	`OlmoLayerNorm` / `Olmo2RMSNorm`	Non-affine variant (no weight parameter!)
StableLM	`StableLmLayerNorm`	Standard
Granite	`GraniteRMSNorm`	Standard

Liger Implementation

Exact Source Files

Area	File	What it contains

Core RMSNorm op	src/liger_kernel/ops/rms_norm.py	Triton forward kernels, backward kernels, Python launch wrappers, and LigerRMSNormFunction, the custom PyTorch autograd function.
Transformer module wrapper	src/liger_kernel/transformers/rms_norm.py	nn.Module wrapper LigerRMSNorm and model-specific variants for Gemma, Gemma2, Gemma3, OLMo2, GLM-4, Qwen3Next.
Functional API	src/liger_kernel/transformers/functional.py	Functional helper liger_rms_norm(...) that directly calls LigerRMSNormFunction.apply(...).
Fused add + RMSNorm op	src/liger_kernel/ops/fused_add_rms_norm.py	Triton implementation for residual add fused with RMSNorm.
Fused add + RMSNorm module	src/liger_kernel/transformers/fused_add_rms_norm.py	nn.Module wrapper for fused residual add + RMSNorm.
Monkey patch integration	src/liger_kernel/transformers/monkey_patch.py	Replaces Hugging Face model RMSNorm classes/modules with Liger versions.
RMSNorm tests	test/transformers/test_rms_norm.py	Correctness tests versus PyTorch reference implementations, including dtype, in-place, affine, and DTensor cases.
Benchmark infra	benchmark/README.md	Liger benchmark framework layout and visualization workflow.

$$y_i = \frac{x_i}{\mathrm{RMS}(x)} \cdot (w_i + \mathrm{offset}), \quad \mathrm{RMS}(x) = \sqrt{\frac{1}{N}\sum_k x_k^2 + \epsilon}$$

- Offset = 1 for Gemma, 0 for LLama.
- Stores only rstd extra for backward

- Two implementations: Normal and block

  NORMAL IMPLEMENTATION

- Forward Normal implementation: A block allocated for each hidden dimension, each program handles one Hidden dimension (1 row).
- Backward Normal implementation: Each program handles a set of rows (number of rows = total_rows/ sm_count, -> sm_count calculated at launch according to architecture). We handle multiple rows to decrease working memory.
  Without: num_columns*num_rows*fp32 — store each row's dW and accumulate at the end.
  With: sm_count*num_columns*fp32 (num_columns ~ Hidden dimension)
- Branching in code according to which model we are using controls: offset and dtype of operands.

  BLOCK IMPLEMENTATION (When n_rows > 32k and n_cols < 256→ 1 program per row will result in huge overhead on dispatcher/scheduler without meaningful work per program)

- We handle [Block_rows, Block_cols] per program instead of [1, Block_cols]
- Has Explicit support for XPU (INTEL GPU LINE)

Unsloth Implementation

Exact Source Files

File	Purpose
`unsloth/kernels/rms_layernorm.py`	The Triton kernels + `torch.autograd.Function` + `fast_rms_layernorm` entry point + `patch_rms_layernorm` / `unpatch_rms_layernorm`
`unsloth/kernels/utils.py`	`calculate_settings()` — picks `BLOCK_SIZE` and `num_warps`. Also `MAX_FUSED_SIZE`.
`unsloth/kernels/__init__.py`	Re-exports `fast_rms_layernorm`, `patch_rms_layernorm`, `unpatch_rms_layernorm`
`unsloth/models/llama.py`	Calls `fast_rms_layernorm(self.input_layernorm, hidden_states)` inside `LlamaDecoderLayer_fast_forward`
`unsloth/models/gemma.py`	Iterates `named_modules()`, finds `GemmaRMSNorm`, ensures `variance_epsilon` exists, dispatches with the `gemma=True` flag
`unsloth_zoo/patching_utils.py`	`patch_layernorm` (sister utility used for non-RMS LayerNorm) — referenced for context

- Liger has more complex implementation - has a blocked implementation as well (not present in sloth for RMSNorm)
- Both Liger and Unsloth Calculate Block_size Manually (dosent use triton.autotune) (Potentially cna improve here)
- torch.compile breaks for them on FASTRMSNorm - therefore no optimisations from pytorch on this- they are still yet to fix the issue

- Few things different from Liger:
    - 1) Strides are marked astl.constexpr
    - 2) tl.hueristics used after jit.compile - mainly for 1/”true” normalisation
    - 3) Not model specific dtype casting unlike Liger
    - 4) Unsloth explicitly pins eps to float32 via tl.full((), eps, tl.float32) to avoid type-promotion bugs on HIP/ROCm. Liger doesn't do this — potentially fragile on AMD GPUs in _CASTING_MODE_NONE.
- But for inference Unsloth is using simple pytorch since we are processing (1, hidden_state) at a time and torch.compile does a better job handling the fusion with other operations in such case, allocating a prog. Can pass pre allocated buffers instead of allocating a new one everytime for each RMS call.
- Has a fall back code in pytorch if the triton kernel fails to run there, eg running on AMD
- Dosent fuse residual with RMSNorm (like flash Attention) since during backward reuses dY’s memory to put dX (saving reallocation time)

Liger implementation is way better than Unsloth. Can take few things from unsloth like explicit eps dtype casting etc.

Scope of Contribution:

1) Using Liger implementation with some parts of unsloth to handle edge cases
2) Fusion of RMSNorm with lm_head since RMSNorm is always called once at the end (Still yet to check if there are any benefits to that)
3) All RMSNorm calls regardless of where it is called (per Q/K_head, pre/post MLP etc) go through the same function – may be can be optimised (either scope of fusion or different num_warps/BLOCK_SIZE depending on function) - yet to look into
4) Give control based on memory-heavy/compute-heavy